FOOD DELIVERY PLATFORM

Service Communication Guide

Comprehensive Guide to Inter-Service Communication in Microservices

REST, OpenFeign, Service Discovery, Resilience Patterns & Best Practices

Project: food-delivery-platform (Maven Multi-Module)
Version: 1.0.0-SNAPSHOT
Focus: Service Communication Architecture
Technologies: Spring Cloud, Eureka, OpenFeign, Resilience4j

Table of Contents

1. Introduction to Inter-Service Communication

Inter-service communication refers to the mechanisms and protocols that allow microservices to exchange data and coordinate their actions. In a microservices architecture, applications are broken down into small, independent services that work together to deliver business functionality.

The Communication Challenge:

- Network Latency: Calls take milliseconds instead of nanoseconds
- Partial Failures: One service may be down while others are running
- Data Serialization: Objects must be converted to wire format (JSON, XML, etc.)
- Service Discovery: Services need to find each other dynamically
- Security: Communication must be authenticated and authorized
- Observability: Calls must be traced and monitored

2. Why Microservices Need Inter-Service Communication

Distributed Nature of Microservices

Microservices are designed to be independently deployable and scalable units. Each service owns a specific business capability and its own data. To deliver complete business functionality, these services must collaborate.

Business Process Orchestration

Complex business processes often span multiple services. A single user action may trigger a chain of service calls.

```
Customer places order
    ↓
Order Service validates customer (calls customer-service)
    ↓
Order Service validates restaurant (calls restaurant-service)
    ↓
Order Service processes payment (calls payment-service)
    ↓
Order Service sends notification (calls notification-service)
    ↓
Order Service assigns delivery partner (calls delivery-partner-service)
```

3. Synchronous vs Asynchronous Communication

Aspect	Synchronous	Asynchronous
Pattern	Request-Response	Event-Driven
Blocking	Yes	No
Coupling	Temporal	Loose
Complexity	Low	High
Feedback	Immediate	Delayed
Scalability	Limited	High
Use Case	Real-time operations	Background processing

4. REST-Based Service-to-Service Communication

REST (Representational State Transfer) is an architectural style for distributed systems. It uses standard HTTP methods (GET, POST, PUT, DELETE) to operate on resources identified by URIs.

REST Principles:

- Resource-Based: Everything is a resource with a unique URI
- Uniform Interface: Standard HTTP methods and status codes
- Stateless: Each request contains all necessary information
- Client-Server: Separation of concerns
- Cacheable: Responses can be cached to improve performance

5. HTTP Clients in Spring Boot

Feature	RestTemplate	WebClient	OpenFeign
Type	Imperative	Reactive	Declarative
Blocking	Yes	No	Yes
Status	Maintenance	Recommended	Recommended
Learning Curve	Low	High	Low
Service Discovery	Manual	Manual	Built-in
Load Balancing	Manual	Manual	Built-in
Best For	Simple calls	Reactive apps	Microservices

OpenFeign Example:

```
@FeignClient(name = "customer-service", path = "/customers")
public interface CustomerClient {
    @GetMapping("/{id}")
    CustomerResponseDto getCustomer(@PathVariable("id") Long id);
}
```

6. Service URL Management Challenges

The Hardcoded URL Problem:

In traditional monolithic applications, service URLs are often hardcoded. This approach is fragile to environment changes and doesn't work in containerized deployments.

Service Discovery Pattern:

Service discovery allows services to dynamically find each other without hardcoded URLs. Services register with a service registry (Eureka, Consul), and clients query the registry for service locations.

In Food Delivery Platform:

- Uses Netflix Eureka for service discovery
- Services register on startup via @EnableDiscoveryClient
- Feign clients use service names instead of URLs
- Automatic load balancing across instances

7. Handling Network Latency and Failures

The Fallacies of Distributed Computing:

- The network is reliable - It's not. Networks fail.
- Latency is zero - It's not. Calls take time.
- Bandwidth is infinite - It's not. Networks have limits.
- The network is secure - It's not. Security must be added.
- Topology doesn't change - It does. Services move.

Failure Handling Patterns:

- Try-Catch: Basic exception handling
- Fallback: Provide alternative behavior
- Circuit Breaker: Stop calling failing services
- Retry: Retry transient failures

8. Timeout and Retry Considerations

Why Timeouts Matter:

Without timeouts, a slow or unresponsive service can block threads indefinitely, cause thread pool exhaustion, cascade failures to other services, and create system-wide outages.

Types of Timeouts:

- Connect Timeout: Time to establish connection
- Read Timeout: Time to receive response after connection
- Overall Timeout: Total time for the entire operation

Retry Strategies:

- Fixed Delay: Retry after fixed time interval
- Exponential Backoff: Increase delay between retries
- Custom Retry: Configure max attempts and backoff strategy

9. Designing Resilient Service Calls

The Resilience Patterns:

- Circuit Breaker: Stop calling failing services
- Retry: Retry transient failures
- Timeout: Don't wait forever
- Bulkhead: Limit resource usage
- Fallback: Provide alternative when service fails
- Cache: Reduce calls to services

Circuit Breaker States:

```
CLOSED (normal) → OPEN (tripped) → HALF_OPEN (testing) → CLOSED
```

10. When Synchronous Communication is Appropriate

Use Cases for Synchronous Communication:

- Immediate Response Required: Payment validation, authentication
- Simple Request-Response: Get customer details, fetch restaurant info
- Data Consistency Required: Operations that need immediate confirmation
- Low Latency Requirements: Real-time updates, interactive workflows

When to Avoid Synchronous Communication:

- Long-Running Processes: Order processing, report generation
- High Throughput Requirements: Event ingestion, analytics
- Loose Coupling Needed: Independent service evolution
- Event-Driven Workflows: Order lifecycle events, notification triggers

11. Implementation in Food Delivery Platform

The Food Delivery Platform uses synchronous REST-based communication with OpenFeign as the primary HTTP client. The order-service acts as an orchestrator, coordinating calls to multiple services.

Feign Clients in Order Service:

Feign Client	Target Service	Endpoint
CustomerClient	customer-service	GET /customers/{id}
RestaurantClient	restaurant-service	GET /restaurants/{id}
PaymentClient	payment-service	POST /payments/process
NotificationClient	notification-service	POST /notifications
DeliveryPartnerClient	delivery-partner-service	POST /delivery-partners/assign

Enabling Feign Clients:

```
@SpringBootApplication
@EnableDiscoveryClient
@EnableFeignClients
public class OrderServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderServiceApplication.class, args);
    }
}
```

12. Best Practices and Recommendations

General Best Practices:

- Use DTOs for Inter-Service Communication - Never expose entities directly
- Implement Service Discovery - Never hardcode service URLs
- Add Timeouts - Always configure connect and read timeouts
- Implement Retry with Backoff - Retry transient failures
- Use Circuit Breakers - Protect against cascading failures
- Add Logging and Monitoring - Track latency and error rates
- Use Idempotent Operations - Safe to retry without side effects

OpenFeign Best Practices:

- Use Interfaces - Define Feign clients as interfaces
- Define Separate DTOs - Client-specific DTOs in calling service
- Use Path Variables Correctly - Match endpoint patterns
- Add Request Interceptors - For authentication headers
- Configure Timeouts per Service - Different timeouts for different services

Resilience Best Practices:

- Combine Multiple Patterns - Circuit breaker + retry + timeout
- Implement Meaningful Fallbacks - Return cached data or default values
- Monitor Circuit Breaker States - Track open/close transitions
- Test Failure Scenarios - Chaos engineering and fault injection